

# Arhitectura calculatoarelor

## Curs 6

### 6. Procesorul cu cicluri multiple de execuție – calea de date

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

# Obiective

1. Generalități
2. Proiectarea multi-ciclu

# 6.1 Generalități

## Pași de proiectare

1. Analiza la nivel RTL (pornind de la ISA).
2. Stabilirea componentelor pentru calea de date.
3. Asamblarea componentelor (cu ajutorul RTL).
4. Descrierea RTL concret (pe baza RTL abstract și a căii de date).
5. Implementarea logicii de control (pe baza RTL concret).

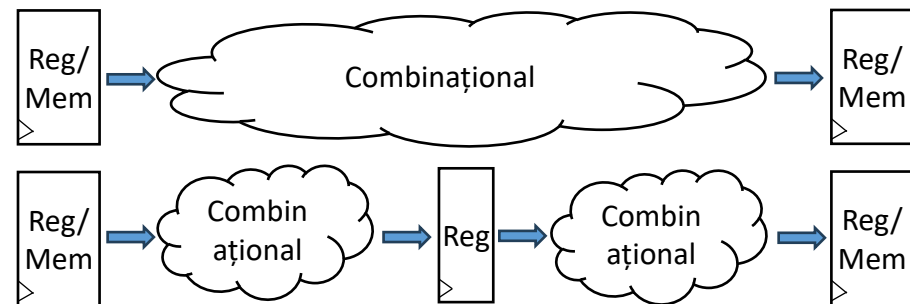
## Motivație

Dezavantaje la varianta cu ciclu unic:

- Perioadă de ceas mare datorită căii critice;
- Consum mare de resurse.

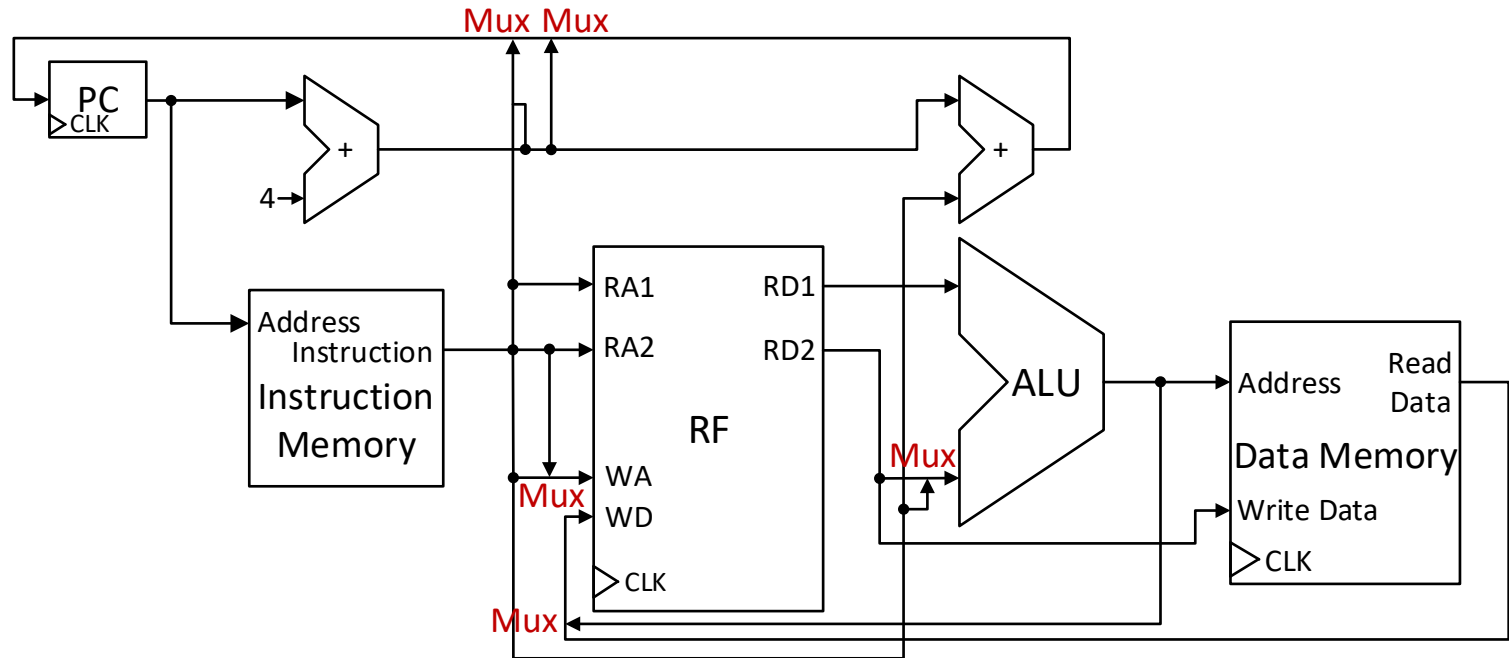
## Avantajele soluției multi-ciclu

- Perioadă de ceas redusă – se secționează partea combinațională prin introducerea de registre;
- Instrucțiuni cu număr de cicluri diferit;
- Se refolosesc unitățile funcționale.



## 6.2 Proiectarea multi-ciclu

### Varianta cu ciclu unic (schiță) – punct de start în proiectare



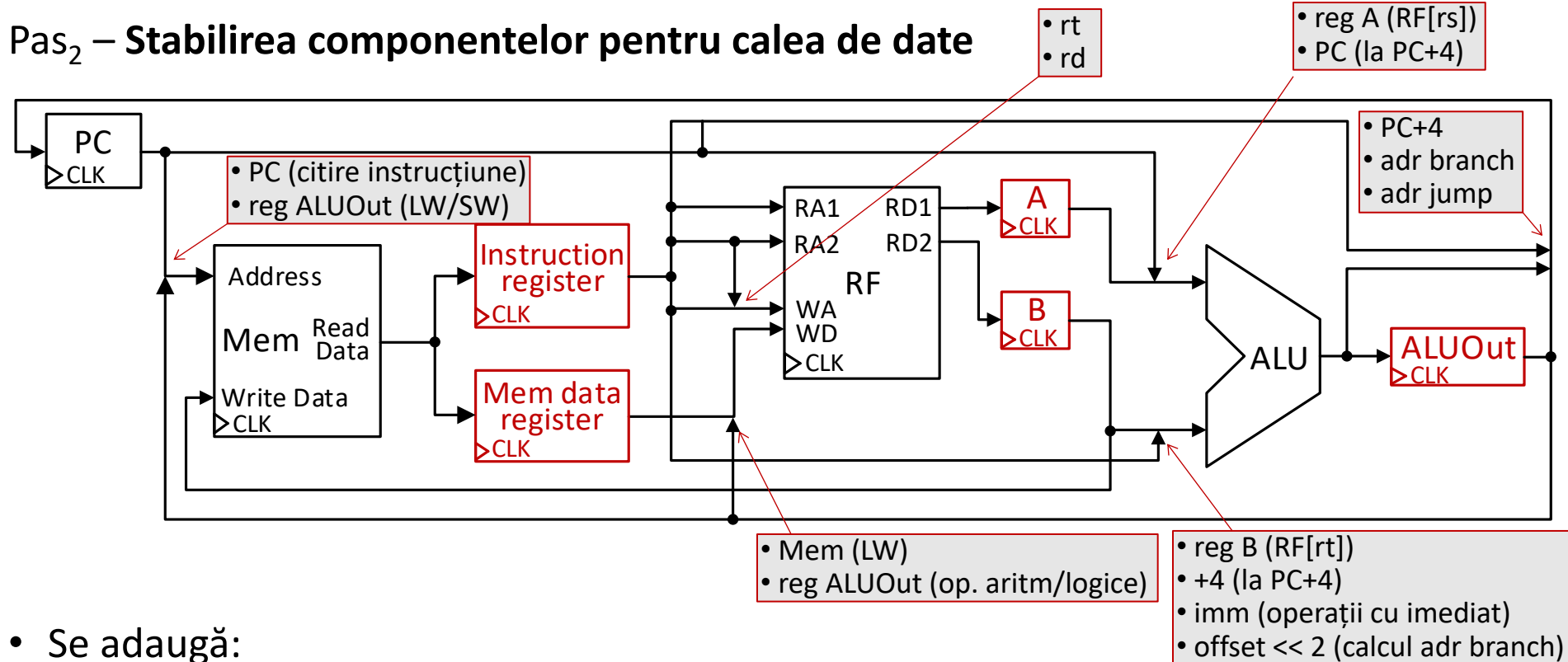
### Considerente de proiectare

- Se secționează instrucțiunile în pași simpli (1 pas / ciclu ceas) de complexitate echilibrată.
  - Maxim o singură unitate funcțională complexă la un pas.
- Se memorează rezultatele unui pas la finalul acestuia => se introduc registre interne invizibile pentru programator.
- Se refolosesc unitățile funcționale: ALU folosit pentru PC+4 și calculul adresei de branch.
- Memorie comună pentru instrucțiuni și date.

## 6.2 Proiectarea multi-ciclu

Pas<sub>1</sub> – Analiza la nivel RTL pornind de la ISA: ca la ciclu unic.

Pas<sub>2</sub> – **Stabilirea componentelor pentru calea de date**



• Se adaugă:

- Instruction register (IR), Memory data register (MDR), registrele A și B, registrul ALUOut, care **nu sunt vizibile programatorului**.

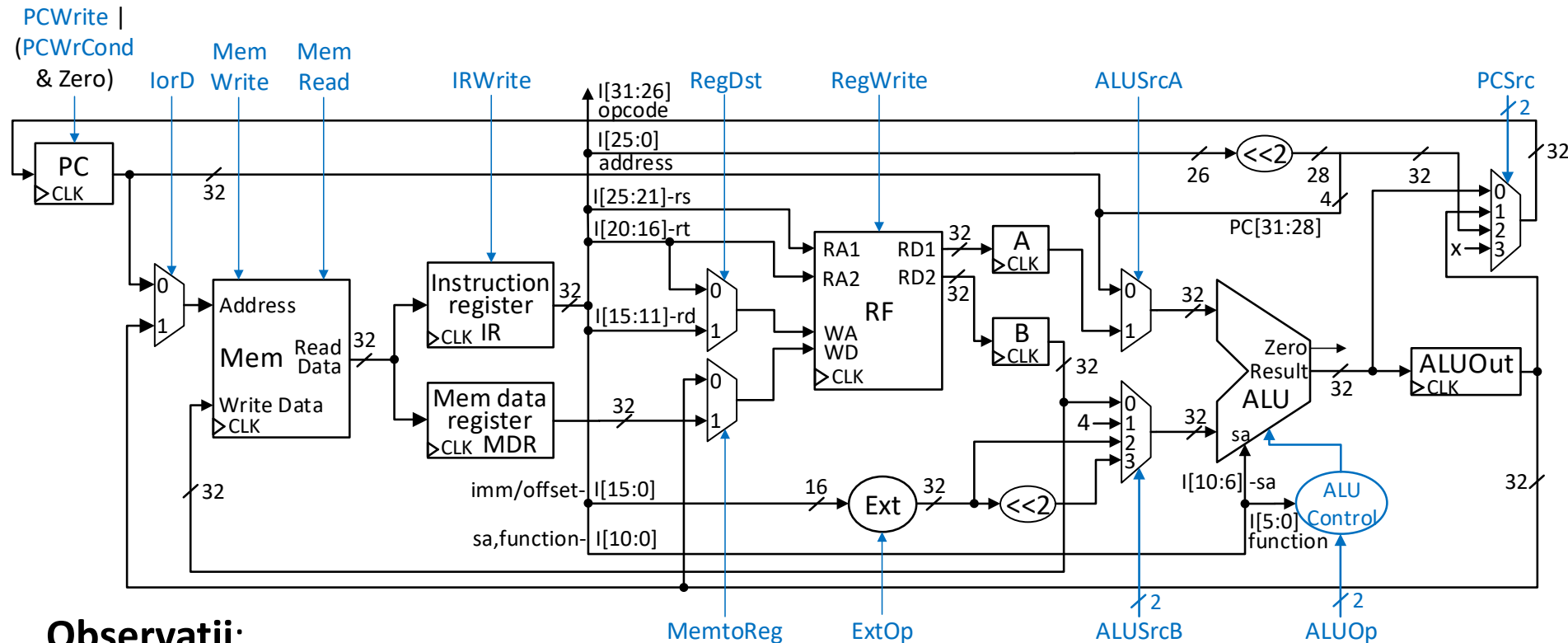
• Se refolosesc:

- ALU pentru calcul PC+4 și adresa de branch;
- Memoria este la comun pentru date și instrucțiuni.

## 6.2 Proiectarea multi-ciclu

### Pas<sub>3</sub> – Asamblarea componentelor cu ajutorul RTL

- Se specifică și semnalele de control



#### Observații:

- Scrierea în memorie este sincronă și se realizează cu comanda `MemWrite=1`.
- Citirea din memorie este asincronă și se realizează cu comanda `MemRead=1`, altfel memoria păstrează pe ieșirea `Read Data` datele de la ultima citire.
- Scrierea în registrele `MDR`, `A`, `B`, `ALUOut` se realizează la fiecare ciclu de ceas.

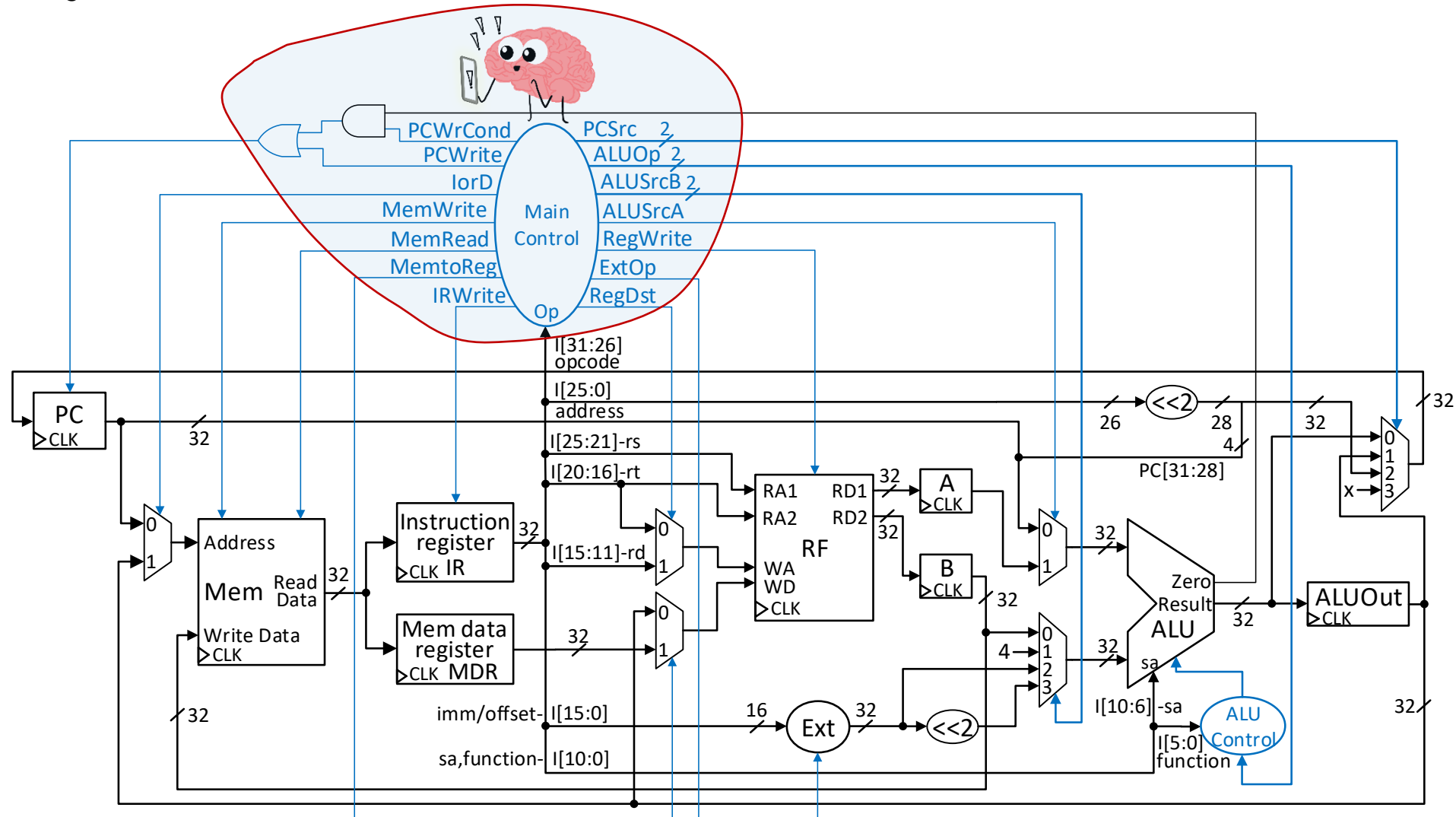
## 6.2 Proiectarea multi-ciclu

### Semnificația semnalelor de control

| Nume        | Efect                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| PCWrite     | Activează scrierea în registrul PC.                                                                                                               |
| PCWriteCond | Activează scrierea în registrul PC dacă flag-ul Zero=1.                                                                                           |
| IorD        | Adresa în memorie este dată de PC pentru instrucțiuni (=0) sau de registrul ALUOut pentru date (=1).                                              |
| MemWrite    | Activează scrierea sincronă în memorie.                                                                                                           |
| MemRead     | Activează citirea asincronă din memorie.                                                                                                          |
| MemtoReg    | Registrul destinație primește valoarea din registrul MDR (=1) sau din registrul ALUOut (=0).                                                      |
| IRWrite     | Activează scrierea în registrul de instrucțiuni IR.                                                                                               |
| RegDst      | Registrul destinație din RF este dat de câmpul <i>rt</i> (=0) sau <i>rd</i> (=1) din reg IR.                                                      |
| ExtOp       | Se face extindere cu semn (=1) sau fără semn (=0).                                                                                                |
| RegWrite    | Activează scrierea sincronă în registrul destinație din RF.                                                                                       |
| ALUSrcA     | Sursa operandului A de la ALU este PC (=0) sau registrul A (=1).                                                                                  |
| ALUSrcB     | Sursa operandului B de la ALU este registrul B (=00), constanta 4 (=01), imediatul extins (=10) sau offset-ul extins și deplasat la stânga (=11). |
| ALUOp       | Indicație către ALUCtrl: adunare (=00), scădere(=01), tip R (=10), SAU (=11).                                                                     |
| PCSrc       | Sursa PC este rezultatul PC+4 de la ALU (=00), registrul ALUOut cu adresa de branch (=01) sau adresa de salt PC[31:28]    I[25:0]    00 (=10).    |

## 6.2 Proiectarea multi-ciclu

### Pas<sub>3</sub> – Asamblarea componentelor cu ajutorul RTL (**complet**)





## 6.2 Proiectarea multi-ciclu

Pas<sub>4</sub> – Descrierea RTL concret pe baza RTL abstract și a căii de date

### Generalități

- **RTL abstract** – descrie efectul din perspectiva specificațiilor ISA.
  - Ex. **Tip R**:  $RF[rd] \leftarrow RF[rs] \text{ op } RF[rt], PC \leftarrow PC + 4$
- **RTL concret** – descrie pașii din perspectiva implementării hardware.
  - Ex. **Tip R**:

|                                                  |                   |                   |
|--------------------------------------------------|-------------------|-------------------|
| $T_0: IR \leftarrow M[PC], PC \leftarrow PC + 4$ | $\Leftrightarrow$ | Instruction Fetch |
| $T_1: A \leftarrow RF[rs], B \leftarrow RF[rt]$  | $\Leftrightarrow$ | Operand Fetch     |
| $T_2: ALUOut \leftarrow A \text{ op } B$         | $\Leftrightarrow$ | Execute           |
| $T_3: R[rd] \leftarrow ALUOut$                   | $\Leftrightarrow$ | Write-Back        |

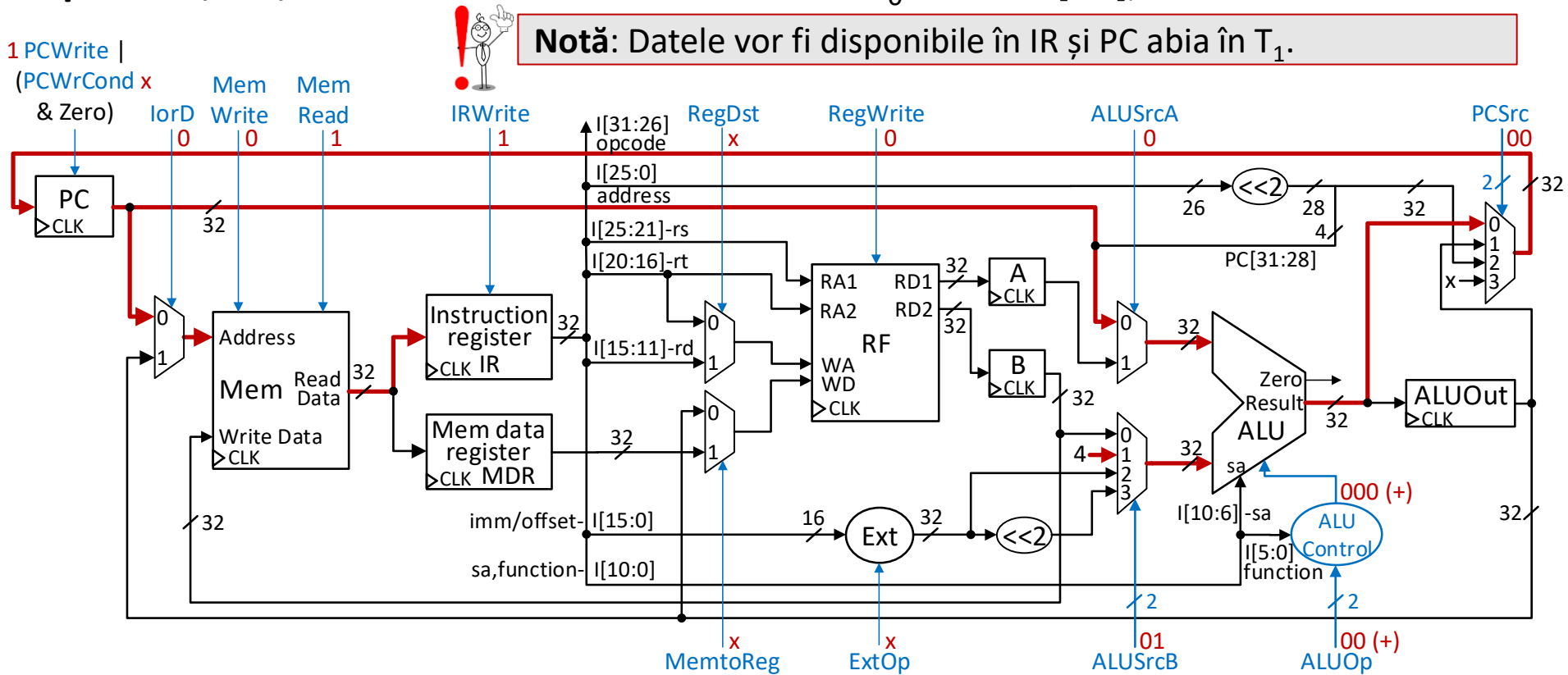
**Obs:** La ciclu unic RTL abstract = RTL concret, nu și la multi-ciclu.

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

## <TipR> \$rd, \$rs, \$rt

RTL concret:  $T_0: IR \leftarrow M[PC], PC \leftarrow PC+4$

**Notă:** Datele vor fi disponibile în IR și PC abia în T<sub>1</sub>.



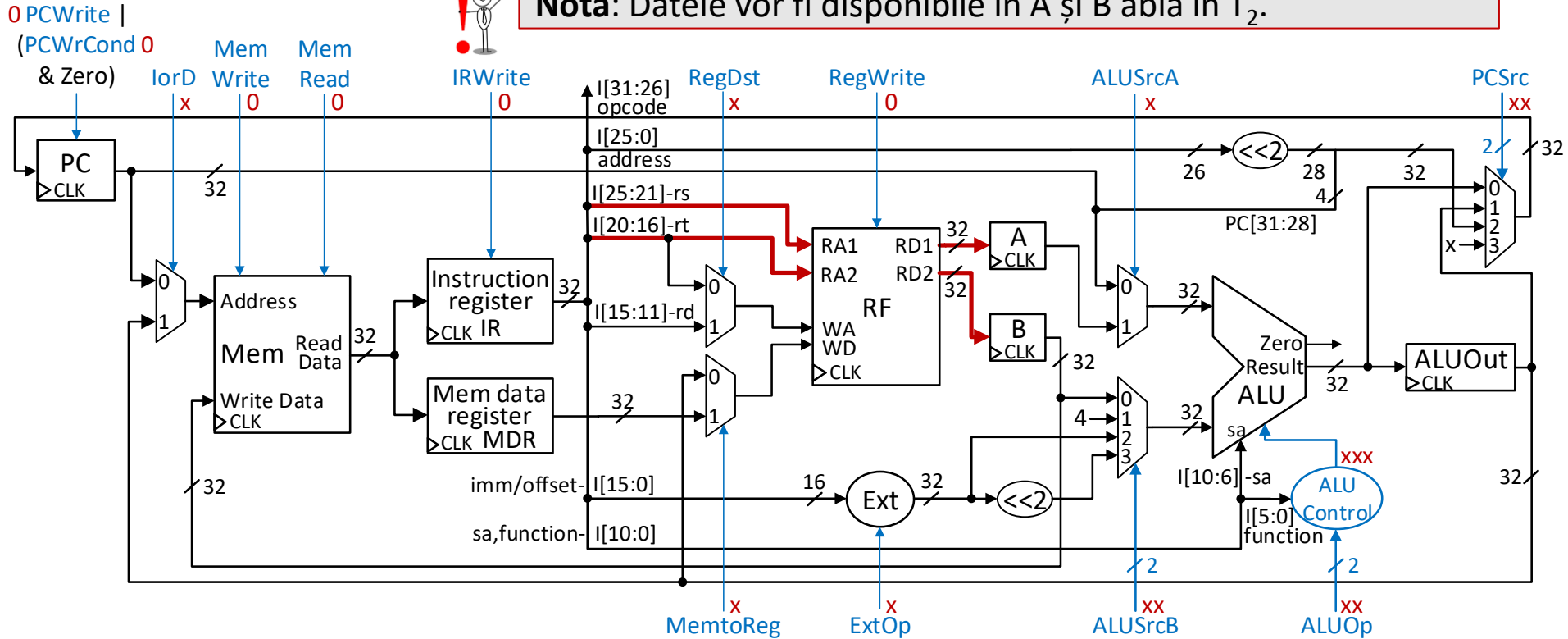
| Pas            | lorD | Mem<br>Read | Mem<br>Write | IR<br>Write | Reg<br>Dst | Mem<br>toReg | Reg<br>Write | Ext<br>Op | ALU<br>SrcA | ALU<br>SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr<br>Cond | PC<br>Write |
|----------------|------|-------------|--------------|-------------|------------|--------------|--------------|-----------|-------------|----------------------------|----------------------|----------------------|--------------|-------------|
| T <sub>0</sub> | 0    | 1           | 0            | 1           | x          | x            | 0            | x         | 0           | 01                         | 00 (+)               | 00                   | x            | 1           |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

<TipR> \$rd, \$rs, \$rt (cont) RTL concret:  $T_1: A \leftarrow RF[rs], B \leftarrow RF[rt]$



**Notă:** Datele vor fi disponibile în A și B abia în  $T_2$ .



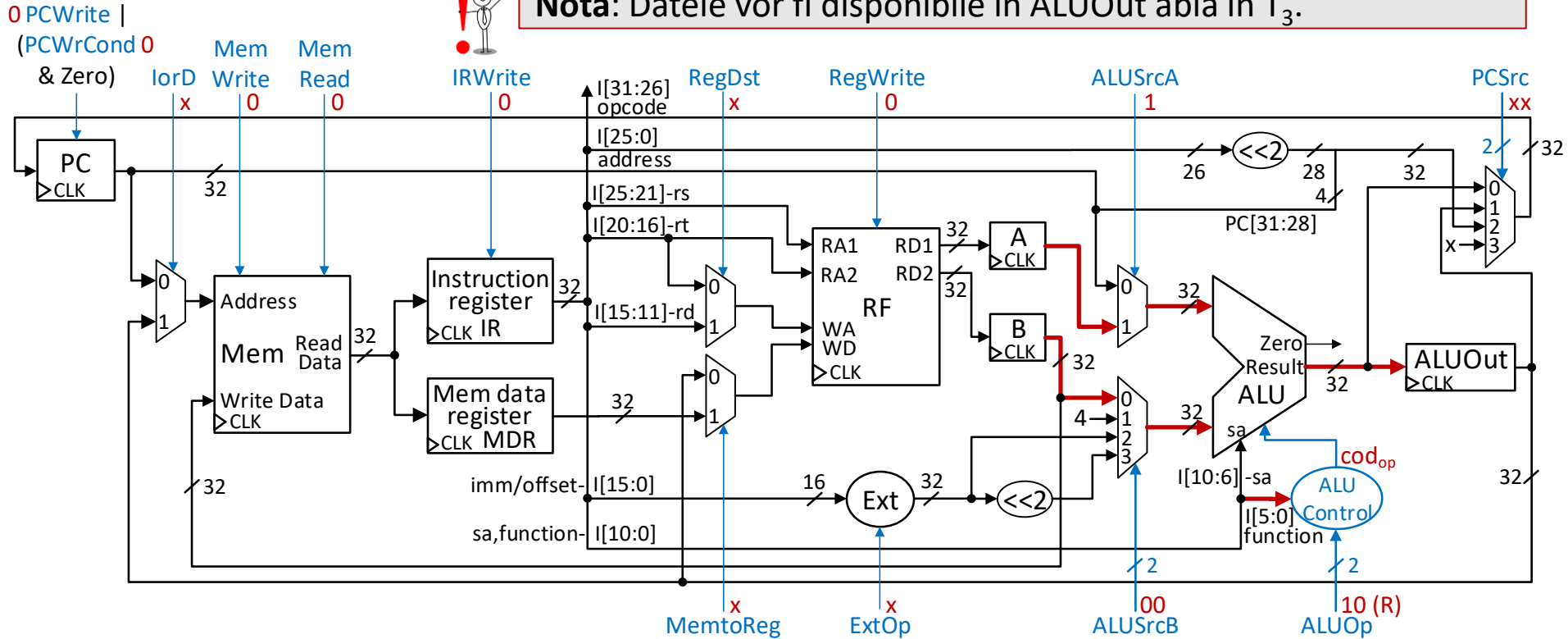
| Pas   | lorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|-------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| $T_0$ | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| $T_1$ | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

<TipR> \$rd, \$rs, \$rt (cont) RTL concret: T<sub>2</sub>: ALUOut ← A op B



**Notă:** Datele vor fi disponibile în ALUOut abia în T<sub>3</sub>.



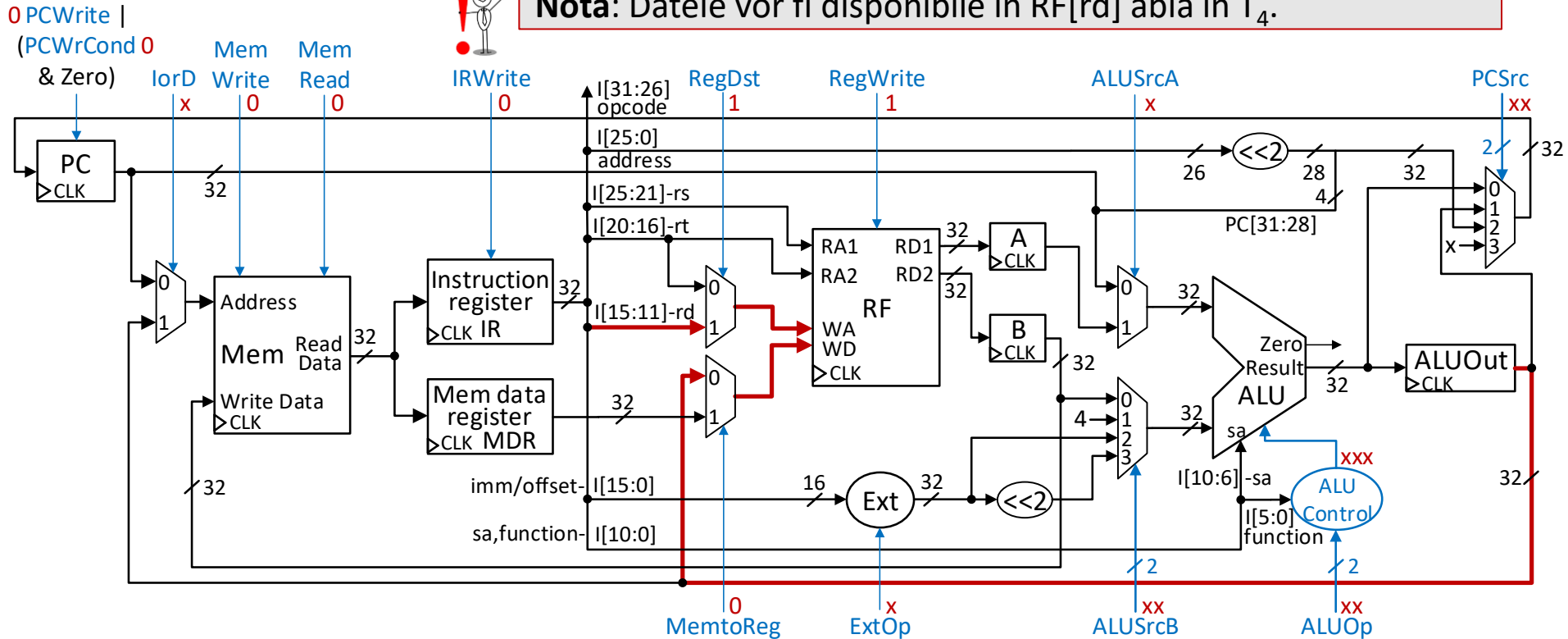
| Pas            | lorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 10 (R)               | xx                   | 0         | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

<TipR> \$rd, \$rs, \$rt (cont) RTL concret: T<sub>3</sub>: RF[rd] ← ALUOut



**Notă:** Datele vor fi disponibile în RF[rd] abia în T<sub>4</sub>.



| Pas            | lorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond       | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x               | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0               | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 10 (R)               | xx                   | 0               | 0        |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 1       | 0         | 1         | x      | x        | xx                      | xx                   | xx                   | 0 <sup>13</sup> | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

<TipR> \$rd, \$rs, \$rt (**Rezumat**)

RTL abstract:  $RF[rd] \leftarrow RF[rs] \text{ op } RF[rt]$ ,  $PC \leftarrow PC + 4$

| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 10 (R)               | xx                   | 0         | 0        |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 1       | 0         | 1         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |



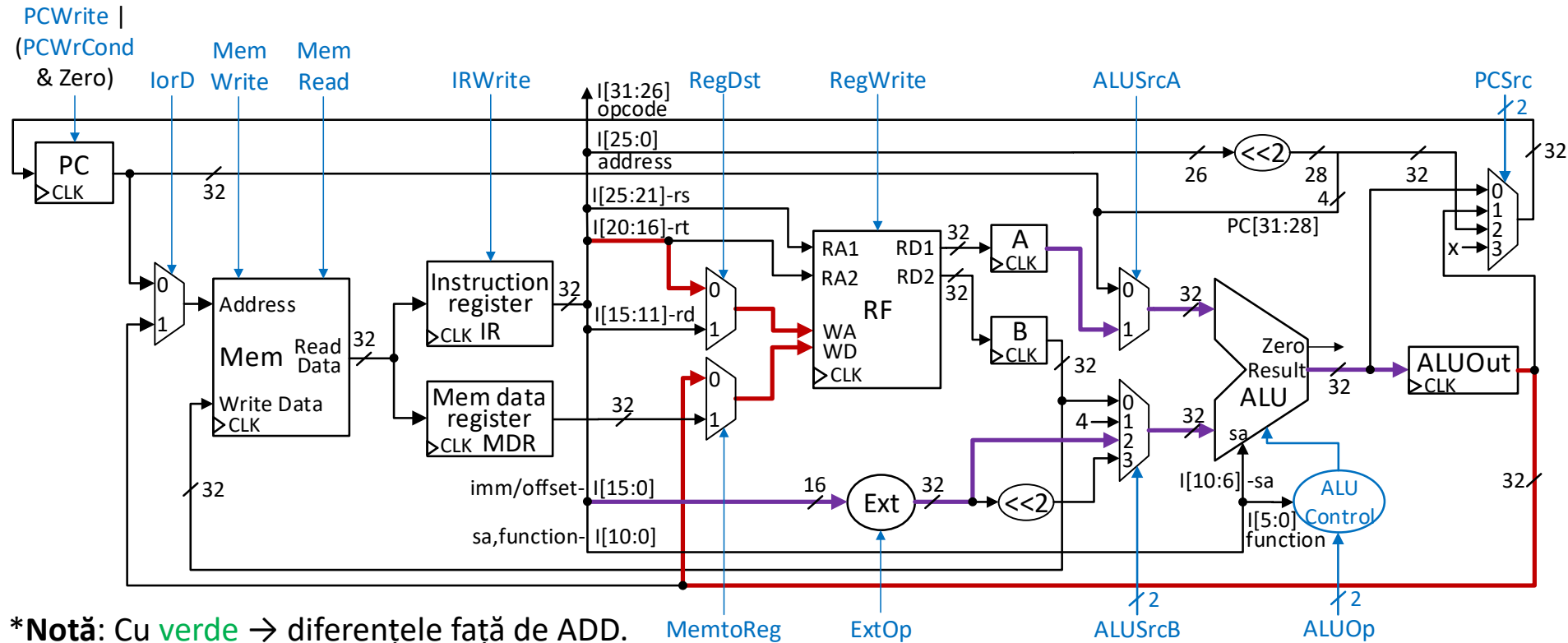
Descriere alternativă

| Pas                 | RTL concret                                  | Semnale relevante                                                                                                                                                                                                                        |
|---------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>      | $IR \leftarrow M[PC]$ , $PC \leftarrow PC+4$ | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB=01}$ , $\overline{ALUOp=(+)}$ , $\overline{PCSrc=00}$ , $\overline{PCWrite}$ |
| T <sub>1</sub>      | $A \leftarrow R[rs]$ , $B \leftarrow R[rt]$  | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                                               |
| TipR:T <sub>2</sub> | $ALUOut \leftarrow A \text{ op } B$          | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB=00}$ , $\overline{ALUOp=(R)}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                     |
| TipR:T <sub>3</sub> | $R[rd] \leftarrow ALUOut$                    | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegDst}$ , $\overline{MemtoReg}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                 |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**ORI \$rt, \$rs, imm**

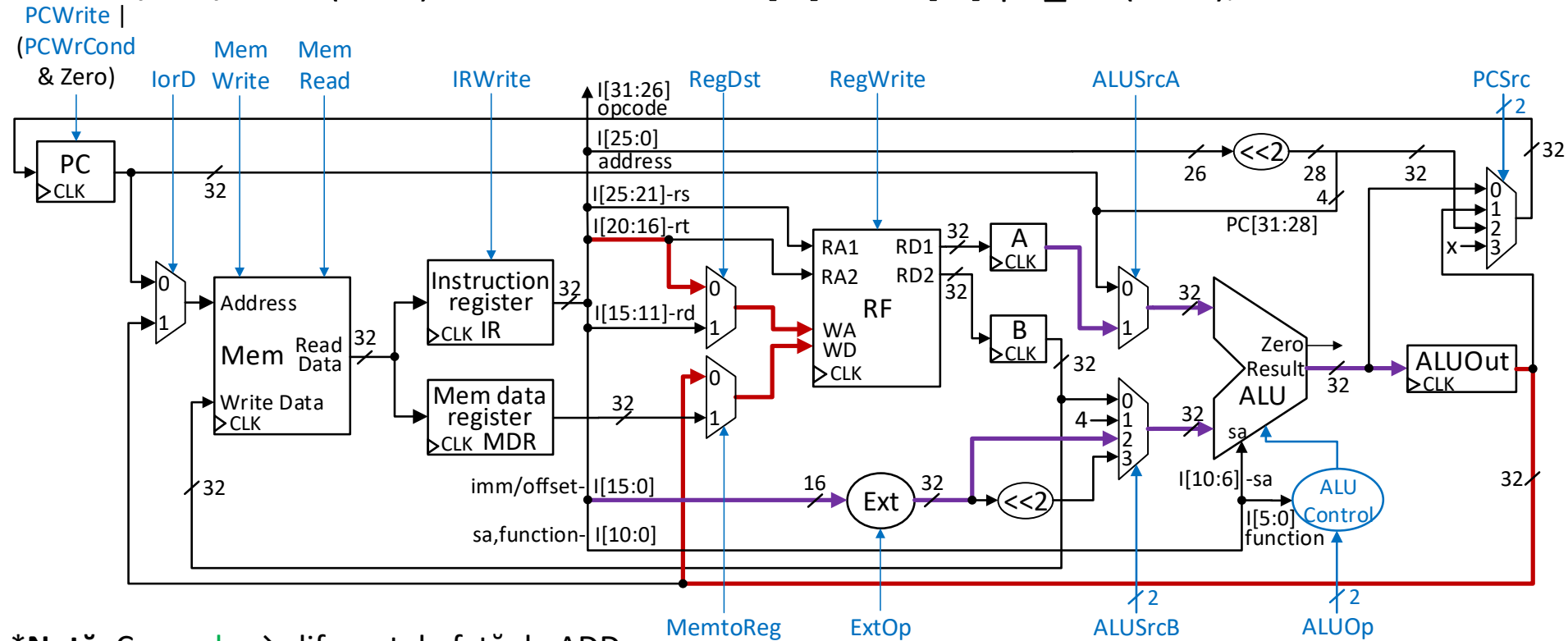
RTL abstract:  $RF[rt] \leftarrow RF[rs] \mid Z\_Ext(imm), PC \leftarrow PC+4$



| Pas               | RTL concret                               | Semnale relevante                                                                                                                                                                                                                                                                               |
|-------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $T_0$             | $IR \leftarrow M[PC], PC \leftarrow PC+4$ | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=01$ , $\overline{ALUOp}=(+)$ , $\overline{PCSrc}=00$ , $\overline{PCWrite}$                                                        |
| $T_1$             | $A \leftarrow R[rs], B \leftarrow R[rt]$  | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                                                                                                      |
| <b>ORI:</b> $T_2$ | $ALUOut \leftarrow A \mid Z\_Ext(imm)$    | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , <b><math>\overline{ExtOp}</math></b> , $\overline{ALUSrcA}$ , <b><math>\overline{ALUSrcB}=10</math></b> , <b><math>\overline{ALUOp}=( )</math></b> , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |
| <b>ORI:</b> $T_3$ | $R[rt] \leftarrow ALUOut$                 | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , <b><math>\overline{RegDst}</math></b> , <b><math>\overline{MemtoReg}</math></b> , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                    |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**ORI \$rt, \$rs, imm (cont)** RTL abstract:  $RF[rt] \leftarrow RF[rs] \mid Z\_Ext(imm), PC \leftarrow PC+4$



**\*Notă:** Cu verde → diferențele față de ADD.

| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond       | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x               | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0               | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 0      | 1        | 10                      | 11 ( )               | xx                   | 0               | 0        |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 0       | 0         | 1         | x      | x        | xx                      | xx                   | xx                   | 0 <sup>16</sup> | 0        |



## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**ADDI \$rt, \$rs, imm**

RTL abstract:  $RF[rt] \leftarrow RF[rs] + S\_Ext(imm)$ ,  $PC \leftarrow PC+4$

\***Notă:** Cu verde → diferențele față de ORI.

| Pas                 | RTL Concret                                  | Semnale relevante                                                                                                                                                                                                  |
|---------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>      | $IR \leftarrow M[PC]$ , $PC \leftarrow PC+4$ | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $ALUSrcB=01$ , $ALUOp=(+)$ , $PCSrc=00$ , $PCWrite$                       |
| T <sub>1</sub>      | $A \leftarrow R[rs]$ , $B \leftarrow R[rt]$  | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                         |
| ADDI:T <sub>2</sub> | $ALUOut \leftarrow A + S\_Ext(imm)$          | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , <b>ExtOp</b> , $\overline{ALUSrcA}$ , $ALUSrcB=10$ , <b>ALUOp=(+)</b> , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |
| ADDI:T <sub>3</sub> | $R[rt] \leftarrow ALUOut$                    | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegDst}$ , $\overline{MemtoReg}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                           |

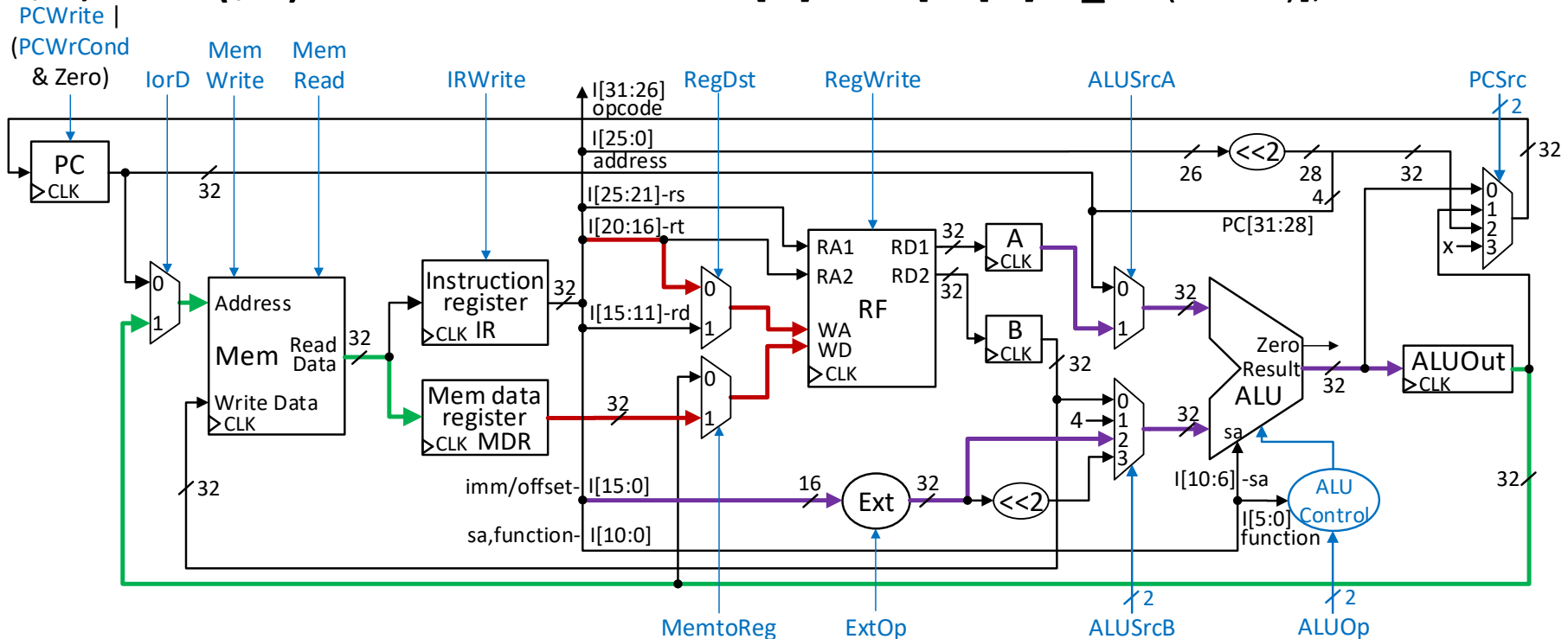
\***Notă:** Cu verde → diferențele față de ORI.

| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op   | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|----------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x        | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x        | x        | xx                      | xx                   | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | <b>1</b> | 1        | 10                      | <b>00 (+)</b>        | xx                   | 0         | 0        |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 0       | 0         | 1         | x        | x        | xx                      | xx                   | xx                   | 0         | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**LW \$rt, offset(\$rs)**

RTL abstract:  $RF[rt] \leftarrow M[RF[rs] + S\_Ext(offset)], PC \leftarrow PC + 4$

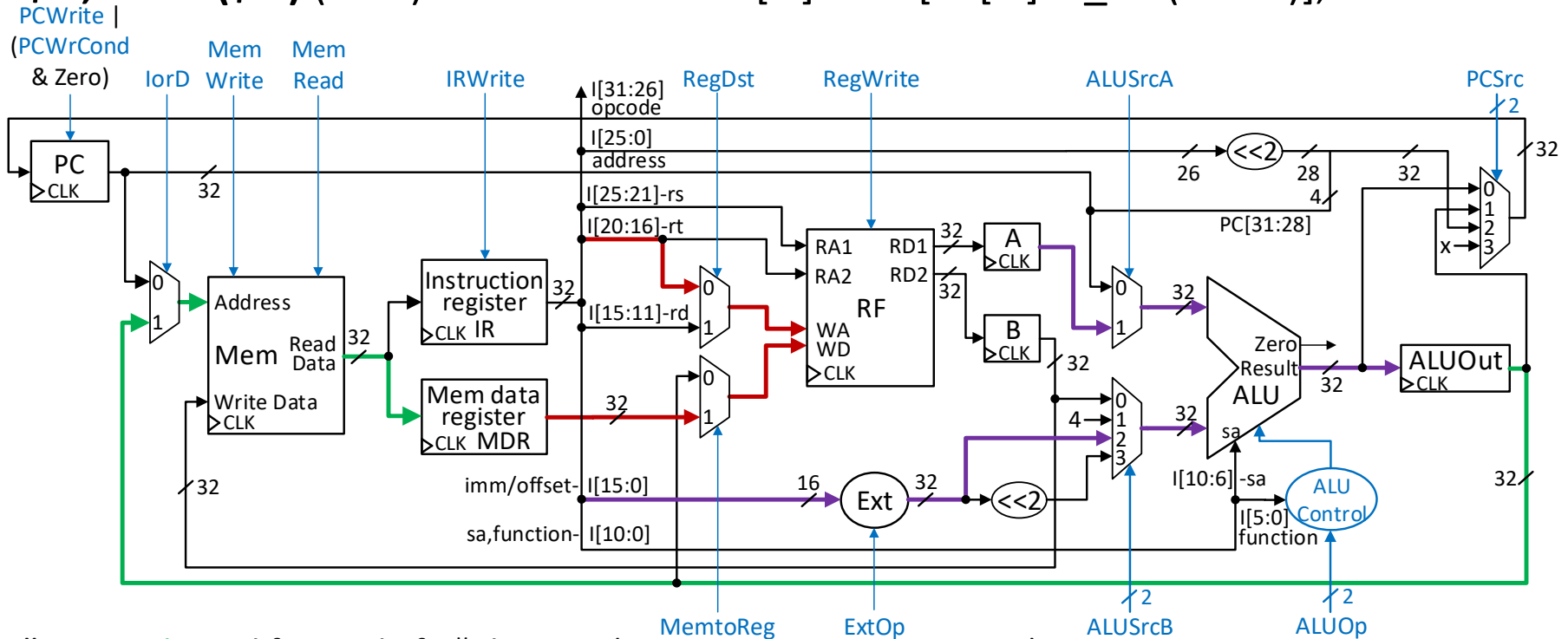


**\*Notă:** Cu verde → diferențele față de ADDI (LW:T<sub>4</sub> comparat cu ADDI:T<sub>3</sub>).

| Pas               | RTL concret                                 | Semnale relevante                                                                                                                                                                                            |
|-------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>    | $IR \leftarrow M[PC], PC \leftarrow PC + 4$ | $\overline{IorD}, \overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{ALUSrcA}, \overline{ALUSrcB}=01, \overline{ALUOp}=+, \overline{PCSrc}=00, \overline{PCWrite}$  |
| T <sub>1</sub>    | $A \leftarrow R[rs], B \leftarrow R[rt]$    | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{PCWrCond}, \overline{PCWrite}$                                                                                  |
| LW:T <sub>2</sub> | $ALUOut \leftarrow A + S\_Ext(offset)$      | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{ExtOp}, \overline{ALUSrcA}, \overline{ALUSrcB}=10, \overline{ALUOp}=+, \overline{PCWrCond}, \overline{PCWrite}$ |
| LW:T <sub>3</sub> | $MDR \leftarrow M[ALUOut]$                  | $\overline{IorD}, \overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{PCWrCond}, \overline{PCWrite}$                                                                 |
| LW:T <sub>4</sub> | $R[rt] \leftarrow MDR$                      | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegDst}, \overline{MemtoReg}, \overline{RegWrite}, \overline{PCWrCond}, \overline{PCWrite}$                                          |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**LW \$rt, offset(\$rs)** (cont) RTL abstract:  $RF[rt] \leftarrow M[RF[rs]+S\_Ext(offset)], PC \leftarrow PC+4$



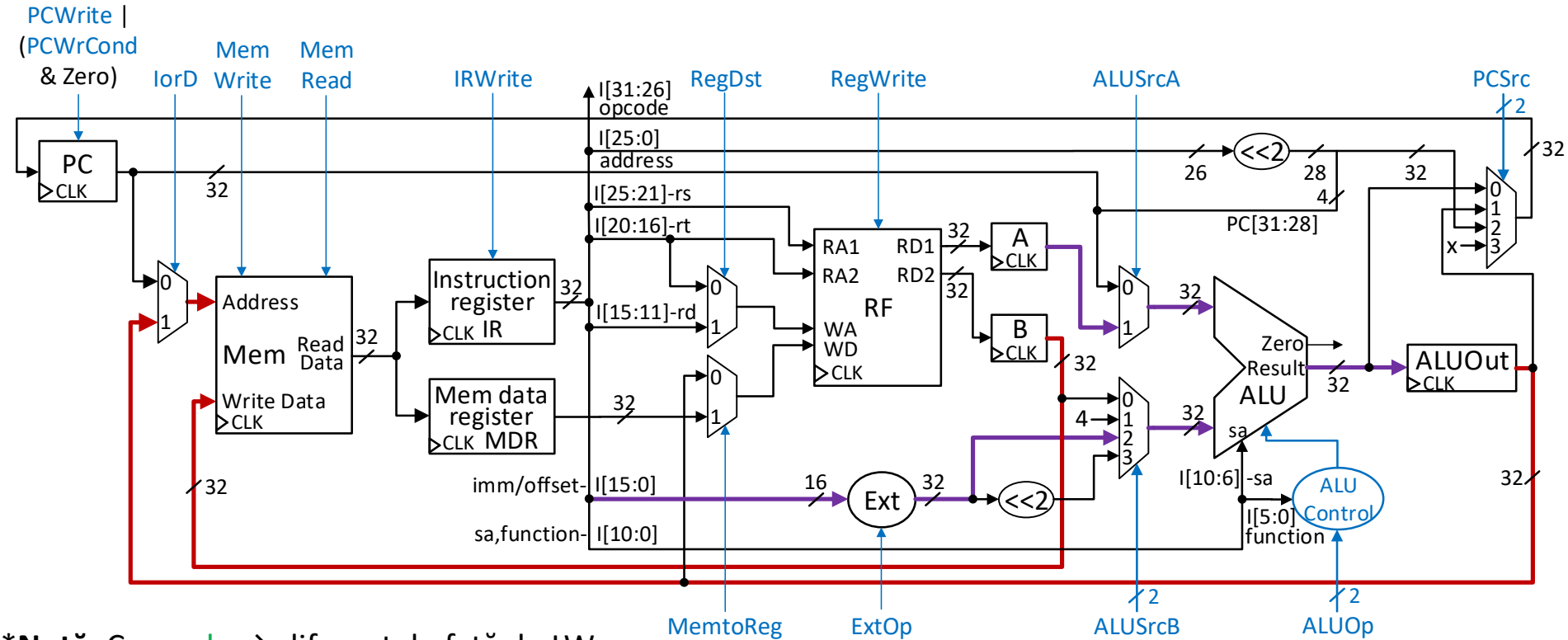
**\*Notă:** Cu verde → diferențele față de ADDI (LW:  $T_4$  comparat cu ADDI:  $T_3$ ).

| Pas            | lorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 1        | 10                      | 00 (+)               | xx                   | 0         | 0        |
| T <sub>3</sub> | 1    | 1        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |
| T <sub>4</sub> | x    | 0        | 0         | 0        | 0       | 1         | 1         | x      | x        | xx                      | xx                   | xx                   | 0         | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**SW \$rt, offset(\$rs)**

RTL abstract:  $M[RF[rs]+S\_Ext(offset)] \leftarrow RF[rt], PC \leftarrow PC+4$

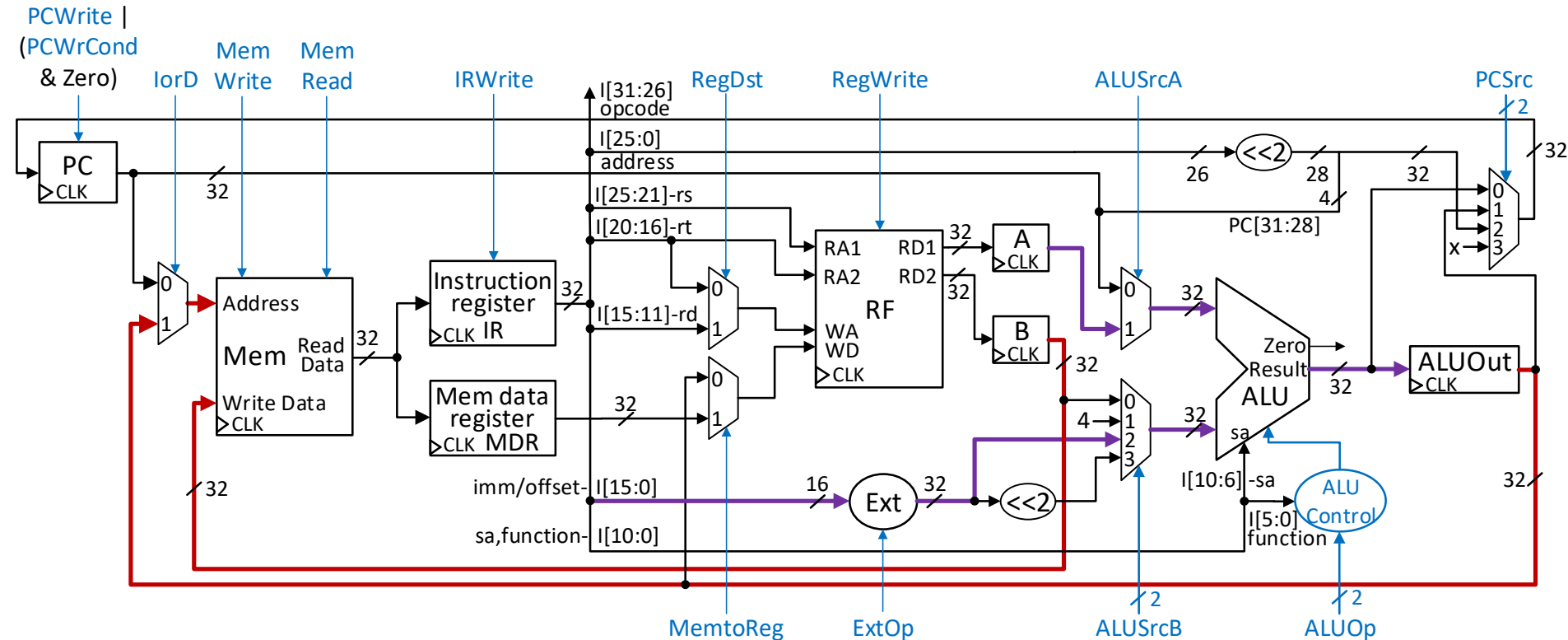


**\*Notă:** Cu verde → diferențele față de LW.

| Pas               | RTL concret                               | Semnale relevante                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>    | $IR \leftarrow M[PC], PC \leftarrow PC+4$ | $\overline{lorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=01$ , $\overline{ALUOp}=+$ , $\overline{PCSrc}=00$ , $\overline{PCWrite}$  |
| T <sub>1</sub>    | $A \leftarrow R[rs], B \leftarrow R[rt]$  | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                                              |
| SW:T <sub>2</sub> | $ALUOut \leftarrow A + S\_Ext(offset)$    | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ExtOp}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=10$ , $\overline{ALUOp}=+$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |
| SW:T <sub>3</sub> | $M[ALUOut] \leftarrow B$                  | $\overline{lorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                          |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**SW \$rt, offset(\$rs)** (cont) RTL abstract:  $M[RF[rs]+S\_Ext(offset)] \leftarrow RF[rt], PC \leftarrow PC+4$

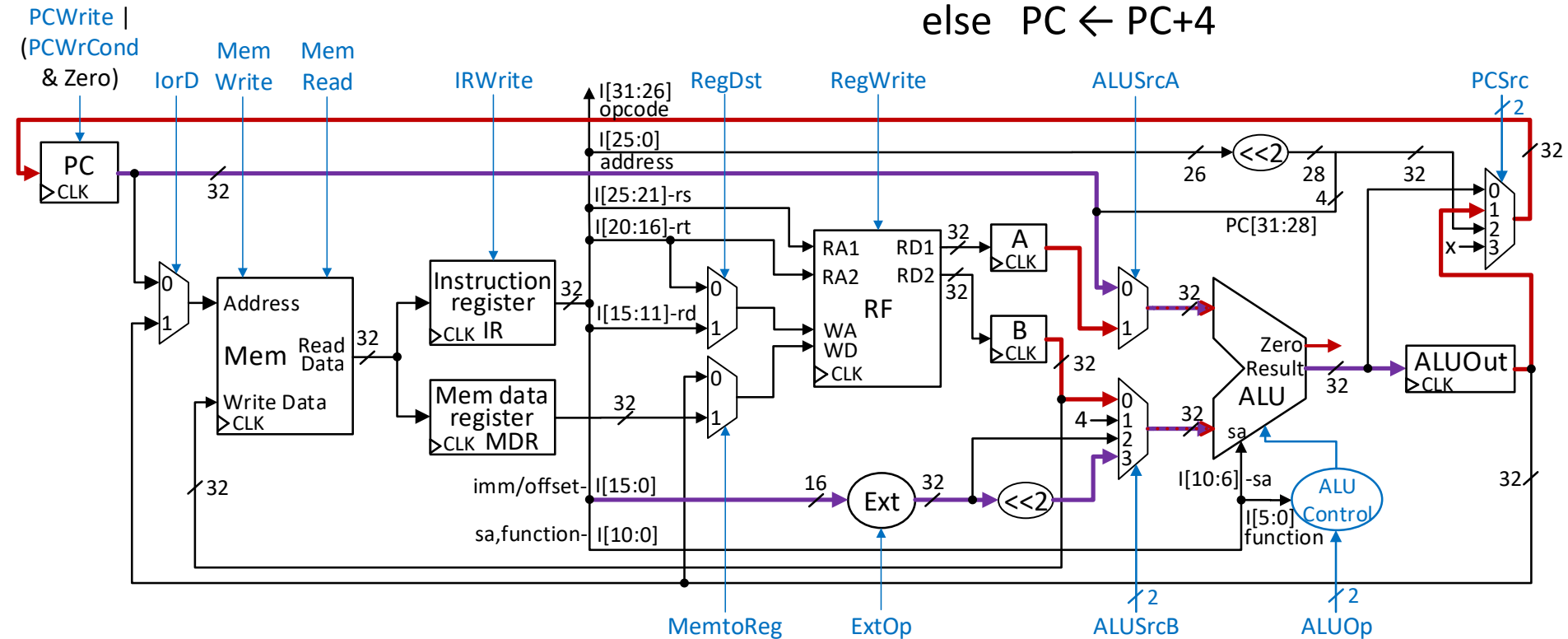


\*Notă: Cu verde → diferențele față de LW.

| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond       | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x               | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0               | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 1        | 10                      | 00 (+)               | xx                   | 0               | 0        |
| T <sub>3</sub> | 1    | 0        | 1         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | xx                   | 0 <sup>21</sup> | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**BEQ \$rs, \$rt, offset**    RTL abstract: if  $RF[rs]=RF[rt]$  then  $PC \leftarrow (PC+4) + S\_Ext(offset) \ll 2$   
 else  $PC \leftarrow PC+4$



| Pas                | RTL concret                                   | Semnale relevante                                                                                                                                                                                                                            |
|--------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>     | $IR \leftarrow M[PC], PC \leftarrow PC+4$     | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=01$ , $\overline{ALUOp}=(+)$ , $\overline{PCSrc}=00$ , $\overline{PCWrite}$     |
| T <sub>1</sub>     | $A \leftarrow R[rs], B \leftarrow R[rt]$      | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$                                                                                                   |
| BEQ:T <sub>2</sub> | $ALUOut \leftarrow PC + S\_Ext(offset) \ll 2$ | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ExtOp}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=11$ , $\overline{ALUOp}=(+)$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$    |
| BEQ:T <sub>3</sub> | $Zero \rightarrow PC \leftarrow ALUOut$       | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=00$ , $\overline{ALUOp}=(-)$ , $\overline{PCSrc}=01$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**BEQ \$rs, \$rt, offset** – Comparație T<sub>1</sub> cu BEQ:T<sub>2</sub>

\***Notă:** Cu **albastru** → diferențele între T<sub>1</sub> și BEQ:T<sub>2</sub>.

| Pas                | RTL concret                                   | Semnale relevante                                                                                                                                                                                                 |
|--------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>     | $IR \leftarrow M[PC], PC \leftarrow PC+4$     | $\overline{IorD}, \overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{ALUSrcA}, \overline{ALUSrcB=01}, \overline{ALUOp=(+)}, \overline{PCSrc=00}, \overline{PCWrite}$     |
| T <sub>1</sub>     | $A \leftarrow R[rs], B \leftarrow R[rt]$      | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{PCWrCond}, \overline{PCWrite}$                                                                                       |
| BEQ:T <sub>2</sub> | $ALUOut \leftarrow PC + S\_Ext(offset) \ll 2$ | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{ExtOp}, \overline{ALUSrcA}, \overline{ALUSrcB=11}, \overline{ALUOp=(+)}, \overline{PCWrCond}, \overline{PCWrite}$    |
| BEQ:T <sub>3</sub> | $Zero \rightarrow PC \leftarrow ALUOut$       | $\overline{MemRead}, \overline{MemWrite}, \overline{IRWrite}, \overline{RegWrite}, \overline{ALUSrcA}, \overline{ALUSrcB=00}, \overline{ALUOp=(-)}, \overline{PCSrc=01}, \overline{PCWrCond}, \overline{PCWrite}$ |

**Observații:**

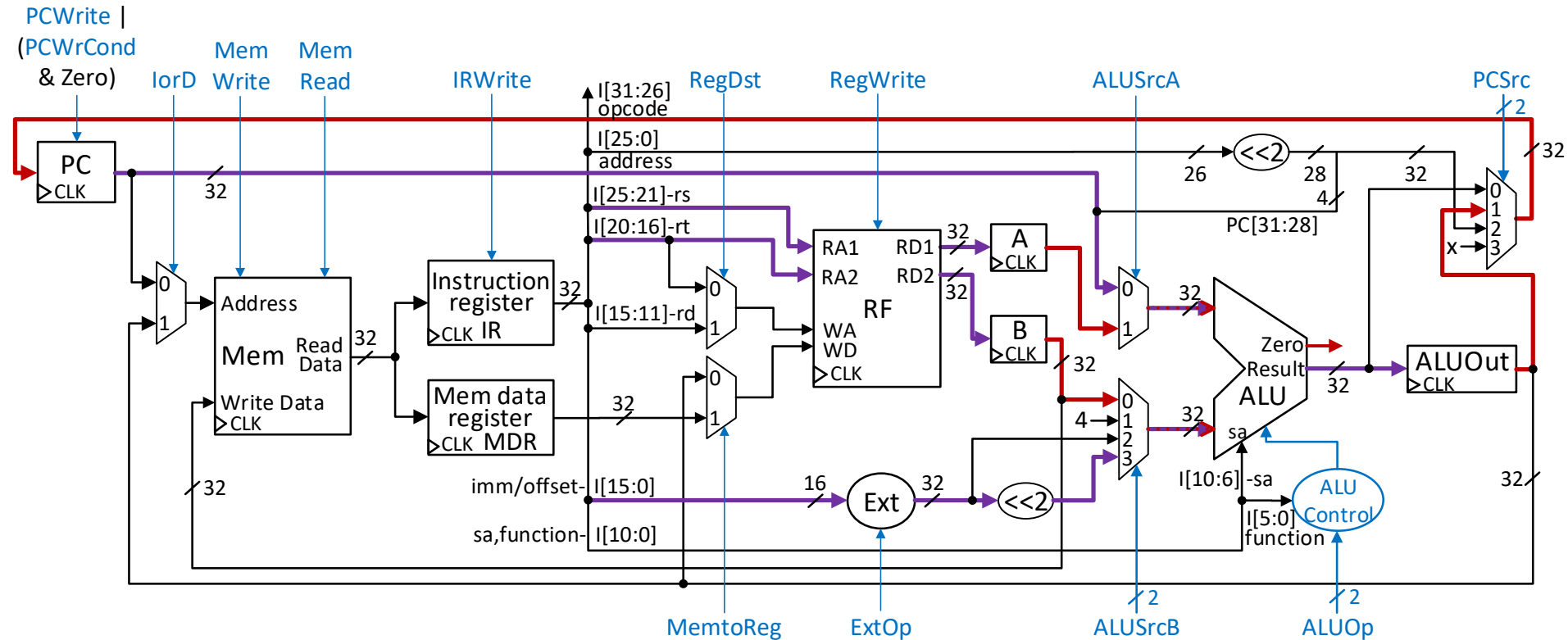
1. BEQ:T<sub>2</sub> are toți operanzii pregătiți după T<sub>0</sub>, respectiv **PC** și **offset**-ul de la IR.
2. BEQ:T<sub>2</sub> are semnale de control fără conflicte cu T<sub>1</sub>.

**Concluzie:**

- BEQ:T<sub>2</sub> și T<sub>1</sub> se pot comasa într-un singur pas.

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**BEQ \$rs, \$rt, offset** – Optimizare prin comasare T<sub>1</sub> cu BEQ:T<sub>2</sub> în T<sub>1</sub>.

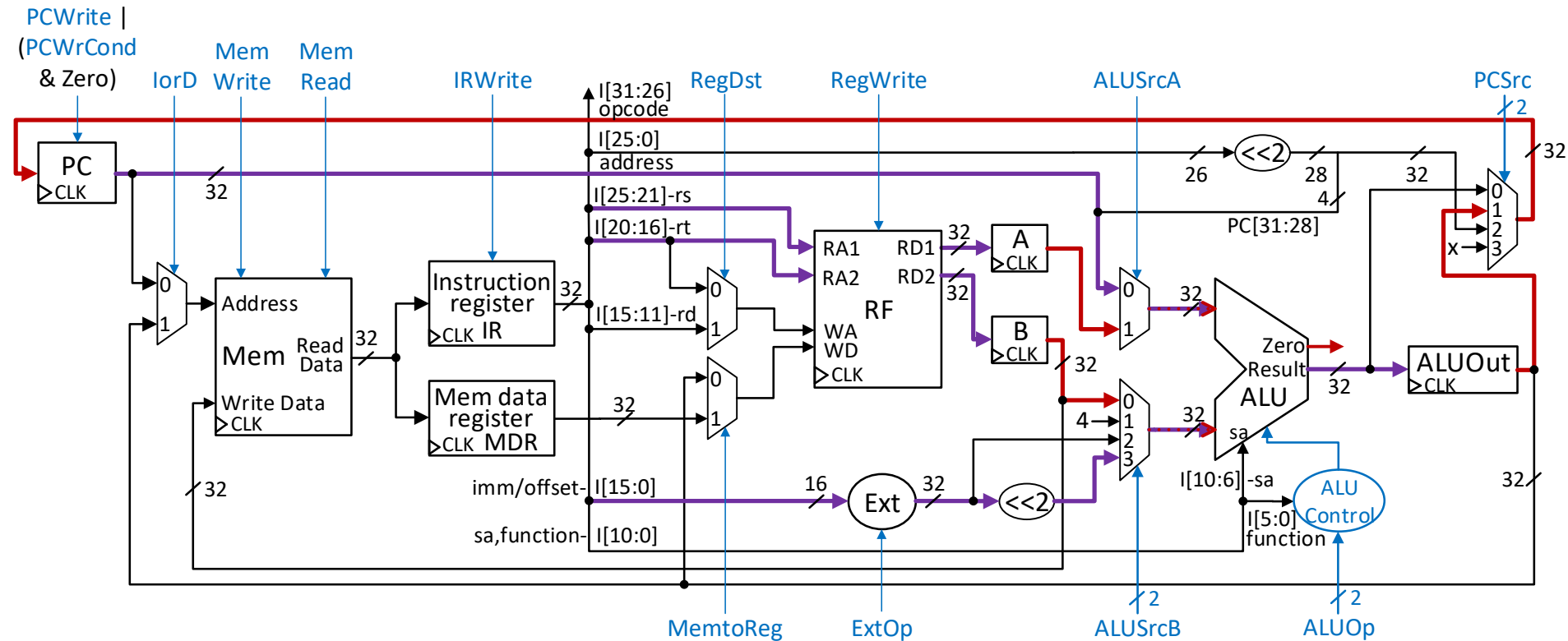


| Pas                | RTL concret                                                                                 | Semnale relevante                                                                                                                                                                                                                            |
|--------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>     | $IR \leftarrow M[PC], PC \leftarrow PC+4$                                                   | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=01$ , $\overline{ALUOp}=(+)$ , $\overline{PCSrc}=00$ , $\overline{PCWrite}$     |
| T <sub>1</sub>     | $A \leftarrow R[rs], B \leftarrow R[rt]$ ,<br>$ALUOut \leftarrow PC + S\_Ext(offset) \ll 2$ | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ExtOp}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=11$ , $\overline{ALUOp}=(+)$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$    |
| BEQ:T <sub>2</sub> | $Zero \rightarrow PC \leftarrow ALUOut$                                                     | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=00$ , $\overline{ALUOp}=(-)$ , $\overline{PCSrc}=01$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |



## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

### BEQ \$rs, \$rt, offset (cont)

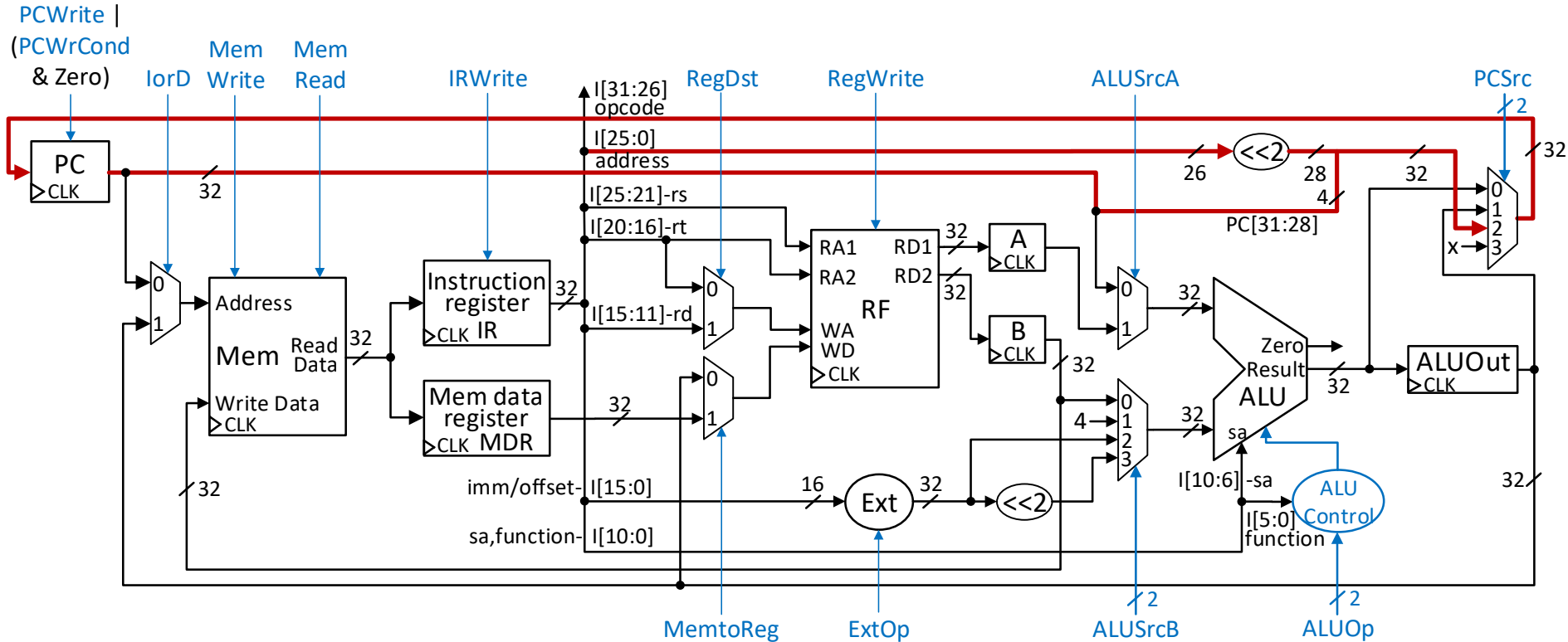


| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 0        | 11                      | 00 (+)               | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 01 (-)               | 01                   | 1         | 0        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**J address**

RTL abstract:  $PC \leftarrow (PC+4)[31:28] || \text{address} || 00$

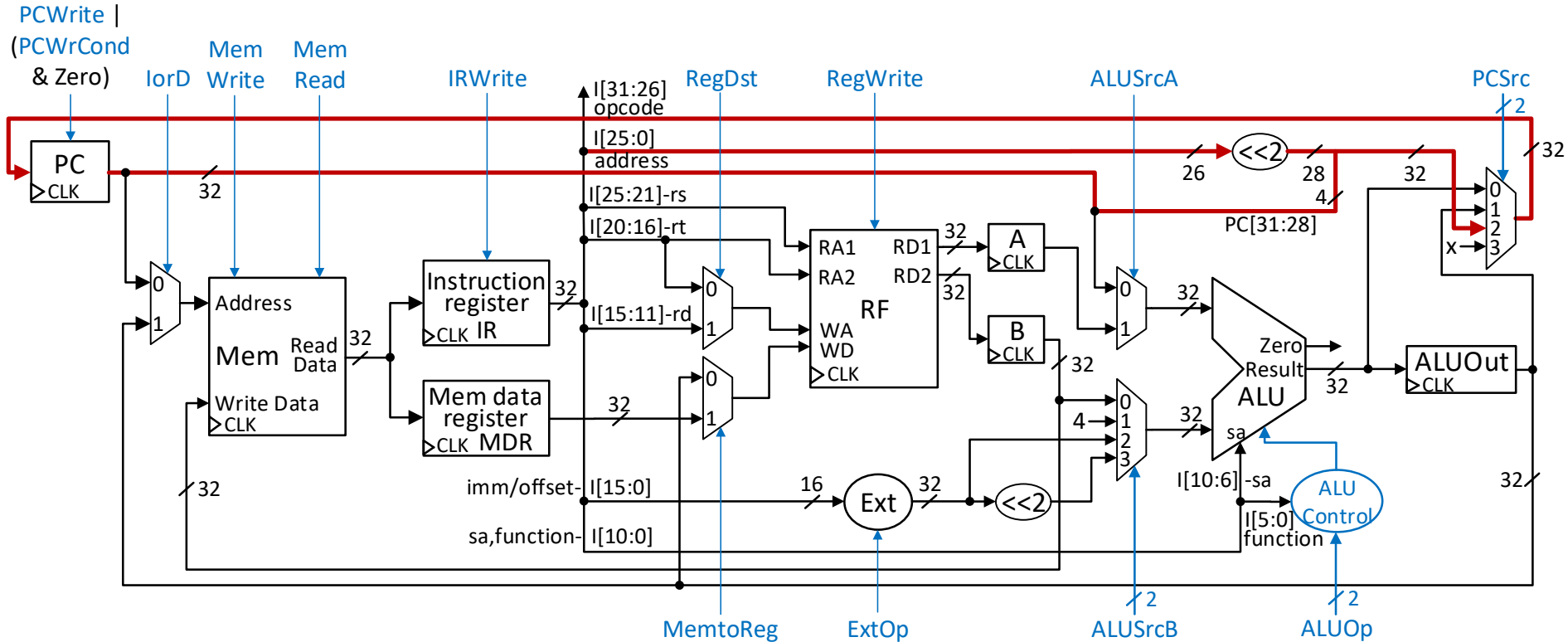


| Pas              | RTL concret                                                                                | Semnale relevante                                                                                                                                                                                                                         |
|------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T <sub>0</sub>   | $IR \leftarrow M[PC], PC \leftarrow PC+4$                                                  | $\overline{IorD}$ , $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=01$ , $\overline{ALUOp}=(+)$ , $\overline{PCSrc}=00$ , $\overline{PCWrite}$  |
| T <sub>1</sub>   | $A \leftarrow R[rs], B \leftarrow R[rt],$<br>$ALUOut \leftarrow PC + S\_Ext(offset) \ll 2$ | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{ExtOp}$ , $\overline{ALUSrcA}$ , $\overline{ALUSrcB}=11$ , $\overline{ALUOp}=(+)$ , $\overline{PCWrCond}$ , $\overline{PCWrite}$ |
| J:T <sub>2</sub> | $PC \leftarrow PC[31:28]    \text{address}    00$                                          | $\overline{MemRead}$ , $\overline{MemWrite}$ , $\overline{IRWrite}$ , $\overline{RegWrite}$ , $\overline{PCSrc}=10$ , $\overline{PCWrite}$                                                                                                |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

**J address (cont)**

RTL abstract:  $PC \leftarrow (PC+4)[31:28] || \text{address} || 00$



| Pas            | lorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub> | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|----------------------|----------------------|-----------|----------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)               | 00                   | x         | 1        |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 0        | 11                      | 00 (+)               | xx                   | 0         | 0        |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                   | 10                   | x         | 1        |

## 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

### Rezumat RTL concret

| Pas                                                                       | Instrucțiuni de tip R                                                                                            | Instr. aritmetice și logice cu imediat                                                   | Instrucțiuni de acces la memorie                               | Instr. de branch                             | Instr. de jump                                           |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------|----------------------------------------------------------|
| T <sub>0</sub> : Instruction Fetch                                        | $IR \leftarrow M[PC], PC \leftarrow PC + 4$ (se realizează simultan)                                             |                                                                                          |                                                                |                                              |                                                          |
| T <sub>1</sub> : Instruction Decode, OperandFetch                         | $A \leftarrow RF[rs], B \leftarrow RF[rt], ALUOut \leftarrow PC + S\_Ext(offset) \ll 2$ (se realizează simultan) |                                                                                          |                                                                |                                              |                                                          |
| T <sub>2</sub> : Execution, address computation, branch / jump completion | $ALUOut \leftarrow A \text{ op } B$                                                                              | ORI: $ALUOut \leftarrow A \mid Z\_Ext(imm)$<br>ADDI: $ALUOut \leftarrow A + S\_Ext(imm)$ | $ALUOut \leftarrow A + S\_Ext(offset)$                         | Zero $\rightarrow$<br>$PC \leftarrow ALUOut$ | $PC \leftarrow PC[31:28] \parallel address \parallel 00$ |
| T <sub>3</sub> : Memory access, R-type completion                         | $RF[rd] \leftarrow ALUOut$                                                                                       | $RF[rt] \leftarrow ALUOut$                                                               | LW: $MDR \leftarrow M[ALUOut]$<br>SW: $M[ALUOut] \leftarrow B$ |                                              |                                                          |
| T <sub>4</sub> : Memory read completion                                   |                                                                                                                  |                                                                                          | LW: $RF[rt] \leftarrow MDR$                                    |                                              |                                                          |

**Obs:** Număr variabil de pași (3-5) în funcție de instrucțiune. În T<sub>1</sub> se decide, pe baza *opcode*, fluxul de control adecvat pentru T<sub>2</sub>.

# 6.2 Proiectarea multi-ciclu – Pas<sub>4</sub>

## Rezumat semnale de control

| Pas            | IorD | Mem Read | Mem Write | IR Write | Reg Dst | Mem toReg | Reg Write | Ext Op | ALU SrcA | ALU SrcB <sub>1:0</sub> | ALUOp <sub>1:0</sub>                 | PCSrc <sub>1:0</sub> | PCWr Cond | PC Write | Fază execuție |
|----------------|------|----------|-----------|----------|---------|-----------|-----------|--------|----------|-------------------------|--------------------------------------|----------------------|-----------|----------|---------------|
| T <sub>0</sub> | 0    | 1        | 0         | 1        | x       | x         | 0         | x      | 0        | 01                      | 00 (+)                               | 00                   | x         | 1        | IF            |
| T <sub>1</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 0        | 11                      | 00 (+)                               | xx                   | 0         | 0        | ID, OF        |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 10 (R)                               | xx                   | 0         | 0        | EX – Tip R    |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 1       | 0         | 1         | x      | x        | xx                      | xx                                   | xx                   | 0         | 0        | WB – Tip R    |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 0/1    | 1        | 10                      | 11 <sub>( )</sub> /00 <sub>(+)</sub> | xx                   | 0         | 0        | EX            |
| T <sub>3</sub> | x    | 0        | 0         | 0        | 0       | 0         | 1         | x      | x        | xx                      | xx                                   | xx                   | 0         | 0        | WB            |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 1        | 10                      | 00 (+)                               | xx                   | 0         | 0        | EX – LW       |
| T <sub>3</sub> | 1    | 1        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                                   | xx                   | 0         | 0        | M – LW        |
| T <sub>4</sub> | x    | 0        | 0         | 0        | 0       | 1         | 1         | x      | x        | xx                      | xx                                   | xx                   | 0         | 0        | WB – LW       |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | 1      | 1        | 10                      | 00 (+)                               | xx                   | 0         | 0        | EX – SW       |
| T <sub>3</sub> | 1    | 0        | 1         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                                   | xx                   | 0         | 0        | M – SW        |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | 1        | 00                      | 01 (-)                               | 01                   | 1         | 0        | EX – BEQ      |
|                |      |          |           |          |         |           |           |        |          |                         |                                      |                      |           |          |               |
| T <sub>2</sub> | x    | 0        | 0         | 0        | x       | x         | 0         | x      | x        | xx                      | xx                                   | 10                   | x         | 1        | EX – J        |

## Concluzii

- Secționare echilibrată a căilor de date prin inserare de registre.
  - Refolosirea componentelor de bază: ALU, memorie comună.
- => mai multe surse la aceeași destinație => se folosesc multiplexoare



căi de date bazate pe multiplexoare

## Probleme propuse

Implementarea altor instrucțiuni (vezi Curs 4, la final – Probleme propuse).

## Referințe

- [1] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, **3rd edition**, Cap. 5.5, Ed. Morgan–Kaufmann, 2004.
- [2] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [3] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.